```
    if (current && (current + ' ' + sentence).length > CHUNK_SIZE) {
      chunks.push(current);
      current = sentence;
    } else {
      current = current ? current + ' ' + sentence : sentence;
    }
  }
  if (current) chunks.push(current);
  return chunks;
}

/* --------------------------- UI rendering --------------------------- */

function buildLanguageOptions() {
  const previous = ui.language.value || 'auto';
  ui.language.textContent = '';

  const auto = document.createElement('option');
  auto.value = 'auto';
  auto.textContent = 'Auto-detect';
  ui.language.append(auto);

  for (const lang of LANGUAGES) {
    const option = document.createElement('option');
    option.value = lang.code;
    const name = lang.name === lang.nativeName ? lang.name : `${lang.name} (${lang.nativeName})`;
    let suffix = ' - checking voices...';
    if (state.voicesLoaded) {
      const count = nativeVoices(lang).length;
      suffix = count > 0 ? ` - ${count} voice${count > 1 ? 's' : ''}` : ' - no voice installed';
    }
    option.textContent = name + suffix;
    ui.language.append(option);
  }
  ui.language.value = getLanguage(previous) || previous === 'auto' ? previous : 'auto';
}

function fillVoiceSelect(list, previousVoice) {
  ui.voice.textContent = '';
  if (list.length === 0) {
    const option = document.createElement('option');
    option.textContent = 'No voice available';
    ui.voice.append(option);
    ui.voice.disabled = true;
    return;
  }

  ui.voice.disabled = false;
  let selected = null;
  for (const voice of list) {
    const option = document.createElement('option');
    option.value = String(state.voices.indexOf(voice));
    option.textContent = `${voice.name} (${voice.lang})`;
    ui.voice.append(option);
    if (previousVoice && voice.name === previousVoice.name && voice.lang === previousVoice.lang) {
```

```
      selected = option;
    }
  }
  if (!selected) {
    const preferred = list.find((voice) => voice.default) || list[0];
    selected = ui.voice.querySelector(`option[value="${state.voices.indexOf(preferred)}"]`);
  }
  if (selected) ui.voice.value = selected.value;
}

function renderLanguageNote(lang) {
  if (lang && lang.engine === 'planned') {
    ui.langNote.textContent =
      `${lang.name}: a dedicated Bloom voice is planned. For now it can only be spoken if your
device has a ${lang.name} voice installed.`;
  } else {
    ui.langNote.textContent = '';
  }
}

function renderDetection() {
  if (ui.language.value !== 'auto') {
    ui.detected.textContent = '';
    return;
  }
  const detection = state.detection;
  if (!detection) {
    ui.detected.textContent = 'Type some text and the language will be detected.';
  } else if (!detection.code) {
    ui.detected.textContent = 'Language not recognised. Please choose one manually.';
  } else {
    const lang = getLanguage(detection.code);
    ui.detected.textContent = detection.reliable
      ? `Detected: ${lang.name}`
      : `Best guess: ${lang.name} (detection for this language is only indicative)`;
  }
}

function refreshVoices() {
  if (!synth) return;

  const previousVoice = state.voices[Number(ui.voice.value)] || null;
  state.voices = synth
    .getVoices()
    .slice()
    .sort((a, b) => a.lang.localeCompare(b.lang) || a.name.localeCompare(b.name));
  if (state.voices.length > 0) state.voicesLoaded = true;

  buildLanguageOptions();

  const lang = currentLanguage();
  let list = [];

  if (state.voices.length === 0) {
    setNotice(
```

```
      state.voicesLoaded
        ? 'No speech voices were found on this device.'
        : 'Loading voices from your device...',
      'warn'
    );
    ui.fallbackWrap.hidden = true;
  } else if (lang) {
    const native = nativeVoices(lang);
    if (native.length > 0) {
      list = native;
      ui.fallbackWrap.hidden = true;
      setNotice(`${native.length} ${lang.name} voice${native.length > 1 ? 's' : ''} available on
this device.`, 'ok');
    } else {
      ui.fallbackWrap.hidden = false;
      setNotice(
        `No ${lang.name} voice is installed on this device, so ${lang.name} cannot be spoken
correctly yet.`,
        'warn'
      );
      if (ui.fallback.checked) list = state.voices;
    }
  } else {
    list = state.voices;
    ui.fallbackWrap.hidden = true;
    setNotice(
      ui.language.value === 'auto'
        ? 'Voices from your device are listed. Choose a language, or type text so it can be
detected.'
        : 'Choose a voice.',
      'info'
    );
  }

  renderLanguageNote(lang);
  renderDetection();
  fillVoiceSelect(list, previousVoice);
  updateControls();
}

function updateControls() {
  const hasText = ui.text.value.trim().length > 0;
  const hasVoice = !ui.voice.disabled && ui.voice.value !== '';
  ui.speak.disabled = !synth || !hasText || !hasVoice || state.mode !== 'idle';
  ui.pause.disabled = state.mode !== 'playing';
  ui.resume.disabled = state.mode !== 'paused';
  ui.stop.disabled = state.mode === 'idle';
}

function updateSliderLabels() {
  ui.rateOut.textContent = `${Number(ui.rate.value).toFixed(1)}x`;
  ui.pitchOut.textContent = Number(ui.pitch.value).toFixed(1);
  ui.volumeOut.textContent = `${Math.round(Number(ui.volume.value) * 100)}%`;
}
```

```
/* ---------------------------- playback ---------------------------- */

function resetPlayback(message) {
  state.mode = 'idle';
  ui.progress.hidden = true;
  setStatus(message);
  updateControls();
}

function playChunk(token) {
  if (token !== player.token || state.mode === 'idle') return;
  if (player.index >= player.chunks.length) {
    resetPlayback('Finished.');
    return;
  }

  const utterance = new SpeechSynthesisUtterance(player.chunks[player.index]);
  if (player.voice) {
    utterance.voice = player.voice;
    utterance.lang = player.voice.lang;
  }
  utterance.rate = Number(ui.rate.value);
  utterance.pitch = Number(ui.pitch.value);
  utterance.volume = Number(ui.volume.value);

  utterance.onstart = () => {
    if (token !== player.token) return;
    ui.progress.max = player.chunks.length;
    ui.progress.value = player.index + 1;
    setStatus(`Speaking part ${player.index + 1} of ${player.chunks.length}...`);
  };
  utterance.onend = () => {
    if (token !== player.token || state.mode !== 'playing') return;
    player.index += 1;
    playChunk(token);
  };
  utterance.onerror = (event) => {
    if (token !== player.token) return;
    if (event.error === 'canceled' || event.error === 'interrupted') return;
    resetPlayback(`Speech error: ${event.error}. Try another voice.`);
  };

  player.current = utterance; // keep a reference so it is not garbage collected mid-speech
  synth.speak(utterance);
}

function startSpeaking() {
  if (ui.speak.disabled) return;
  const chunks = chunkText(ui.text.value);
  if (chunks.length === 0) return;

  synth.cancel();
  const token = ++player.token;
  player.chunks = chunks;
  player.index = 0;
```

```
  player.voice = state.voices[Number(ui.voice.value)] || null;

  state.mode = 'playing';
  ui.progress.hidden = false;
  ui.progress.max = chunks.length;
  ui.progress.value = 0;
  setStatus('Starting...');
  updateControls();

  // A short delay avoids a Chrome bug where speak() right after cancel() is dropped.
  setTimeout(() => playChunk(token), 80);
}

function pauseSpeaking() {
  if (state.mode !== 'playing') return;
  synth.pause();
  state.mode = 'paused';
  setStatus('Paused.');
  updateControls();
}

function resumeSpeaking() {
  if (state.mode !== 'paused') return;
  const token = player.token;
  state.mode = 'playing';
  synth.resume();
  setStatus('Speaking...');
  updateControls();

  // Some mobile browsers cancel the utterance on pause; replay the current part if so.
  setTimeout(() => {
    if (token === player.token && state.mode === 'playing' && !synth.speaking && !synth.pending) {
      playChunk(token);
    }
  }, 300);
}

function stopSpeaking() {
  player.token += 1;
  if (synth) synth.cancel();
  resetPlayback('Stopped.');
}

/* ------------------------------ events ------------------------------- */

function onTextInput() {
  const length = ui.text.value.length;
  ui.count.textContent = `${length} / ${MAX_CHARS}`;

  if (ui.language.value === 'auto') {
    const previousCode = state.detection ? state.detection.code : null;
    state.detection = ui.text.value.trim() ? detectLanguage(ui.text.value) : null;
    const newCode = state.detection ? state.detection.code : null;
    if (previousCode !== newCode) {
      refreshVoices();
```

```
      return;
    }
    renderDetection();
  }
  updateControls();
}

function onLanguageChange() {
  state.detection =
    ui.language.value === 'auto' && ui.text.value.trim() ? detectLanguage(ui.text.value) : null;
  refreshVoices();
}

function insertSample() {
  const lang = currentLanguage() || getLanguage('en');
  ui.text.value = lang.sample;
  onTextInput();
}

async function checkServer() {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), 15000);
  try {
    const response = await fetch('/api/info', {
      signal: controller.signal,
      headers: { Accept: 'application/json' }
    });
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const info = await response.json();
    ui.server.textContent = `Server online - v${info.version}`;
    ui.server.dataset.state = 'ok';
    if (ui.version) ui.version.textContent = info.version;
  } catch {
    ui.server.textContent = 'Server unreachable - browser speech still works';
    ui.server.dataset.state = 'warn';
  } finally {
    clearTimeout(timer);
  }
}

function init() {
  ui.text.maxLength = MAX_CHARS;
  updateSliderLabels();
  checkServer();

  if (!synth) {
    ui.unsupported.hidden = false;
    setNotice('Speech is not available in this browser.', 'warn');
    buildLanguageOptions();
    updateControls();
    return;
  }

  ui.text.addEventListener('input', onTextInput);
  ui.sample.addEventListener('click', insertSample);
```

```
  ui.language.addEventListener('change', onLanguageChange);
  ui.voice.addEventListener('change', updateControls);
  ui.fallback.addEventListener('change', refreshVoices);
  for (const slider of [ui.rate, ui.pitch, ui.volume]) {
    slider.addEventListener('input', updateSliderLabels);
  }
  ui.speak.addEventListener('click', startSpeaking);
  ui.pause.addEventListener('click', pauseSpeaking);
  ui.resume.addEventListener('click', resumeSpeaking);
  ui.stop.addEventListener('click', stopSpeaking);
  window.addEventListener('pagehide', () => synth.cancel());

  if ('onvoiceschanged' in synth) synth.onvoiceschanged = refreshVoices;
  else synth.addEventListener('voiceschanged', refreshVoices);

  refreshVoices();
  // Some browsers never fire voiceschanged when there are no voices.
  setTimeout(() => {
    state.voicesLoaded = true;
    refreshVoices();
  }, 1500);
}

init();
```